

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 4 — CHAINS & LCEL COMPOSITION

Prerequisites: Phases 1–3 done — comfortable with the  pipe, prompts, parsers, and structured output.

Targets: RunnableParallel · RunnableLambda · RunnablePassthrough/.assign · RunnableBranch · debugging

Estimated time for this phase: 80–120 minutes

Where this phase sits

You've built straight-line chains (`prompt` | `model` | `parser`). Real pipelines branch: they run steps **in parallel**, drop in **custom Python**, carry data **alongside** results, and **route** to different sub-chains. This phase gives you those moves — and how to debug them.

Ph 0 ✓ **Environment Setup**

Ph 1 ✓ **LangChain Fundamentals**

Ph 2 ✓ **Working with Models**

Ph 3 ✓ **Prompts & Output Parsers** — templates, few-shot, structured output.

Ph 4 **Chains & LCEL Composition** ← you are here
Parallel execution, custom functions, passthrough/assign, conditional routing, debugging chains.

Ph 5 **Memory & Conversational Apps**

Ph 6–8 **Documents · Embeddings · RAG**

Ph 9–12 **Tools & Agents · LangGraph · Deployment · Capstone**

Phase 4 · Chains & LCEL Composition

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Run several sub-chains at once with `RunnableParallel` ; insert any Python function with `RunnableLambda` ; carry the original input forward and add computed fields with `RunnablePassthrough` / `.assign` ; send inputs down different paths with `RunnableBranch` ; and inspect exactly what flows through a chain when it misbehaves.

In this phase

1. 4.1 — Recap: the sequential chain
2. 4.2 — `RunnableParallel` : do several things at once
3. 4.3 — `RunnableLambda` : drop in your own Python
4. 4.4 — `RunnablePassthrough` & `.assign` : carry data forward
5. 4.5 — `RunnableBranch` : conditional routing
6. 4.6 — Debugging a chain
7. 4.7 — Common errors & troubleshooting
8. 4.8 — Mini practice exercise
9. Terms introduced this phase

4.1 — Recap: the sequential chain

From Phase 1: `a | b | c` builds a `RunnableSequence` — run `a` , feed its output to `b` , then to `c` . Output of each must match the input the next expects. That's a straight line:

```
input → prompt → model → parser → output      (sequential)
```

Every primitive in this phase is just another Runnable you can drop into that line — or use to make the line branch.

4.2 — `RunnableParallel` : do several things at once

Sometimes you want to run multiple independent steps on the *same input* and collect all their results. `RunnableParallel` takes a **dict of Runnables**, gives every one the same input, runs them concurrently, and returns a dict of their outputs under the same keys.

```
input →┐ summary_chain → "..."  
      └─┤ topic_chain  → "..."  
all run AT ONCE
```

```

    ↳ title_chain    → "..."
output = {"summary": "...", "topic": "...", "title": "..."}

```

```

from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel      # 1

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
parser = StrOutputParser()

summary = ChatPromptTemplate.from_template("Summarize in one line: {text}") | model |
parser
title = ChatPromptTemplate.from_template("Give a catchy title for: {text}") | model |
parser

both = RunnableParallel(summary=summary, title=title)      # 2

result = both.invoke({"text": "LangChain lets you swap LLMs without rewriting code."})
# 3
print(result["summary"])
print(result["title"])

```

Line	Explanation
1	Import <code>RunnableParallel</code> from <code>langchain_core.runnables</code> .
2	Two complete sub-chains under keys <code>summary</code> and <code>title</code> . Both receive the same <code>{text: ...}</code> input; they run at the same time, not one-after-another.
3	The result is a dict: <code>{"summary": "...", "title": "..."} . Two model calls, but roughly the wall-clock time of one.</code>

A DICT LITERAL IS A RUNNABLEPARALLEL

Anywhere a Runnable is expected, a plain dict of Runnables is automatically turned into a `RunnableParallel` . So `{"summary": summary, "title": title}` inside a `|` chain works without naming the class. You'll see this shorthand everywhere (especially in RAG, Phase 8).

4.3 — `RunnableLambda` : drop in your own Python

Not every step is a model call. You'll often need plain Python in the middle of a chain — clean a string, do some maths, reshape a dict. `RunnableLambda` wraps any function so it becomes a pipe-able Runnable (you met it briefly in Phase 1).

```

from langchain_core.runnables import RunnableLambda

def word_count(text: str) -> dict:      # an ordinary function

```

```

    return {"text": text, "words": len(text.split())}

count = RunnableLambda(word_count)          # now it's a Runnable

# Use it in a pipe: count the words, THEN ask the model to comment
chain = count | RunnableLambda(lambda d: f"The text has {d['words']} words.")
print(chain.invoke("LangChain makes models swappable"))
# -> The text has 4 words.

```

AUTO-WRAPPING

Inside a `|` chain, a bare function or `lambda` is automatically coerced into a `RunnableLambda` — so `some_runnable | (lambda x: x.upper())` works. Wrapping explicitly is clearer when learning and required when the function is the *first* thing in a chain.

4.4 — `RunnablePassthrough` & `.assign` : carry data forward

A classic problem: a step produces something new, but you also need the *original* input later in the chain. By default each step replaces the data with its own output. `RunnablePassthrough` solves this — it passes the input through unchanged, and its `.assign()` method *adds* new keys while keeping the existing ones.

```

from langchain_core.runnables import RunnablePassthrough, RunnableLambda

# Start with a dict; ADD a "length" key but keep "text"
add_length = RunnablePassthrough.assign(
    length=RunnableLambda(lambda d: len(d["text"]))    # compute from existing key
)

print(add_length.invoke({"text": "hello"}))
# -> {"text": "hello", "length": 5}    original kept, new key added

```

MENTAL MODEL: A DICT FLOWING DOWN THE CHAIN

Many real chains pass a **growing dictionary** from step to step.

`RunnablePassthrough.assign(new_key=...)` is how you bolt another field onto that dict without losing what's already there. This is the exact pattern RAG uses to keep the user's `question` alongside the retrieved `context` (Phase 8).

Plain `RunnablePassthrough()` (no `.assign`) just forwards the input untouched — handy as one branch of a parallel block when you want to keep the raw input around:

```

from operator import itemgetter

prep = {
    "question": RunnablePassthrough(),    # keep the whole input
    "shout":    RunnableLambda(lambda q: q.upper()),    # derive something from it
}

```

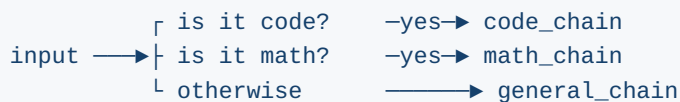
```
print(RunnableParallel(prepare).invoke("hi there"))
# -> {"question": "hi there", "shout": "HI THERE"}
```

TERM: ITEMGETTER

`from operator import itemgetter` gives a tiny helper to pull one key out of a dict in a pipe:
`itemgetter("question")` behaves like `lambda d: d["question"]`. You'll use it to feed just one field of the flowing dict into a sub-chain.

4.5 — RunnableBranch : conditional routing

Sometimes the right sub-chain depends on the input — a coding question should go to a code prompt, a translation request to a translation prompt. `RunnableBranch` is an if/elif/else for chains: a list of `(condition, runnable)` pairs plus a default.



```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableBranch

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
parser = StrOutputParser()

code_chain = ChatPromptTemplate.from_template("Answer as a senior engineer: {q}") | model | parser
general_chain = ChatPromptTemplate.from_template("Answer simply: {q}") | model | parser

router = RunnableBranch(
    (lambda x: "code" in x["q"].lower(), code_chain), # (condition, runnable)
    general_chain, # default (last, no condition)
)

print(router.invoke({"q": "Show me code to reverse a list in Python"})) # -> code_chain
print(router.invoke({"q": "Why is the sky blue?"})) # -> general_chain
```

Piece	Explanation
Each tuple	<code>(condition, runnable)</code> : the condition is a function returning <code>True / False</code> on the input. The first one that's <code>True</code> wins, and its runnable handles the input.
Last argument	The default runnable, used when no condition matched. It has no condition.

SMARTER ROUTING

Conditions can be anything — including the output of a quick classifier chain. A common pattern: a cheap model first labels the input ("code" / "math" / "other"), then `RunnableBranch` routes to the right specialist chain. (For complex branching and loops, Phase 10's LangGraph is the more powerful tool — `RunnableBranch` covers the simple cases.)

4.6 — Debugging a chain

When a chain returns something weird, the question is always "what did each step actually receive and return?" Three reliable ways to find out:

1. Turn on global debug logging

```
from langchain_core.globals import set_debug
set_debug(True)      # prints every step's input/output to the console

chain.invoke({"text": "..."})  # now very verbose
set_debug(False)     # turn it off again
```

This is the fastest way to see the data flowing through, step by step. (`set_verbose(True)` is a lighter version.)

2. Print the chain's structure

```
chain.get_graph().print_ascii()  # draws the pipeline as a diagram
```

Great for confirming the shape of a parallel/branching chain matches what you intended.

3. Tap the data mid-chain

Insert a `RunnableLambda` that prints and passes the value through unchanged — a "probe" you can drop anywhere:

```
from langchain_core.runnables import RunnableLambda

def peek(x):
    print("DEBUG ->", x)  # see exactly what's here
    return x              # pass it on unchanged

chain = prompt | RunnableLambda(peek) | model | parser  # inspect what the model
receives
```

THE REAL SUPERPOWER: LANGSMITH

For anything non-trivial, LangChain's hosted tracing tool **LangSmith** records every step of every run with inputs, outputs, timings, and token counts — no print statements. We give it a proper look in **Phase 11**. For now, `set_debug(True)` and the `peek` probe will carry you a long way.

4.7 — Common errors & troubleshooting

Symptom	Likely cause & fix
A step gets a dict but expected a string (or vice-versa)	Output type of one step doesn't match the next step's input. Insert a <code>RunnableLambda</code> / <code>itemgetter</code> to reshape, or use <code>.assign</code> to keep the dict shape consistent.
<code>RunnableParallel</code> result is missing a key	A branch raised an error or you mistyped the key name. Check each sub-chain runs on its own first.
<code>.assign</code> overwrote my data	<code>.assign(text=...)</code> replaces the existing <code>text</code> key. Use a new key name to add rather than overwrite.
<code>RunnableBranch</code> always hits the default	No condition returned <code>True</code> . Print the input inside the condition (<code>lambda x: print(x) or "code" in x["q"]</code>) to see what you're actually testing.
"object is not callable" / can't pipe	You piped a <i>result</i> instead of a <i>Runnable</i> , or a non-Runnable. Pipe the objects; wrap plain functions in <code>RunnableLambda</code> .
Parallel branches seem to run one-by-one	If branches are CPU-bound Python (not I/O like model calls), Python's threading won't speed them up much. Parallelism shines for concurrent model/network calls.

4.8 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 5)

Task: Build a "smart responder." (1) Use `RunnableParallel` to get, from the same input text, both a one-line `summary` and a detected `language`. (2) Use `RunnablePassthrough.assign` to add a `word_count` field (plain Python) to the flowing dict. (3) Add a `RunnableBranch` that, if the detected language isn't English, routes to a chain that translates the summary to English; otherwise passes it through. Run the whole thing on Gemini and again on Ollama.

Why: it exercises every composition primitive together — parallel, custom Python via lambda/assign, and conditional routing — which is precisely the toolkit you'll lean on to wire up RAG (Phase 8) and agents (Phase 9).

Hints (not the full answer):

- Build and test each piece in isolation first; only then connect them with `|`.
- For the word count, `RunnablePassthrough.assign(word_count=RunnableLambda(lambda d: len(d["text"].split())))`.
- The branch condition reads the `language` field the parallel step produced — so the parallel step must run *before* the branch.
- Stuck on shapes? Drop the `peek` probe (§4.6) between steps to see the dict at each stage.

Terms introduced this phase

Term	One-line meaning
RunnableSequence	A straight-line chain built by <code> </code> ; output of each feeds the next.
RunnableParallel	Runs a dict of Runnables on the same input concurrently; returns a dict of results.
RunnableLambda	Wraps a plain Python function/lambda into a pipe-able Runnable.
RunnablePassthrough	Forwards input unchanged; the foundation for keeping data around.
<code>.assign(key=...)</code>	Adds a computed key to a flowing dict while keeping existing keys.
<code>itemgetter</code>	<code>operator.itemgetter("k")</code> : pull one key out of a dict inside a pipe.
RunnableBranch	If/elif/else for chains: <code>(condition, runnable)</code> pairs + a default.
<code>set_debug</code> / <code>set_verbose</code>	Global switches that log every step's input and output.
<code>get_graph().print_ascii()</code>	Prints the chain's structure as a diagram.
peek probe	A pass-through lambda that prints the value to inspect mid-chain.

PHASE 4 COMPLETE

Your chains can now branch and converge: parallel sub-chains, custom Python steps, data carried forward with passthrough/assign, conditional routing, and three ways to debug the flow. **Next — Phase 5: Memory & Conversational Apps**, where we use the `MessagesPlaceholder` from Phase 3 plus LangGraph's persistence to give a chatbot real memory across turns — the modern replacement for the deprecated `ConversationChain` and `RunnableWithMessageHistory` patterns you'll see in older tutorials.